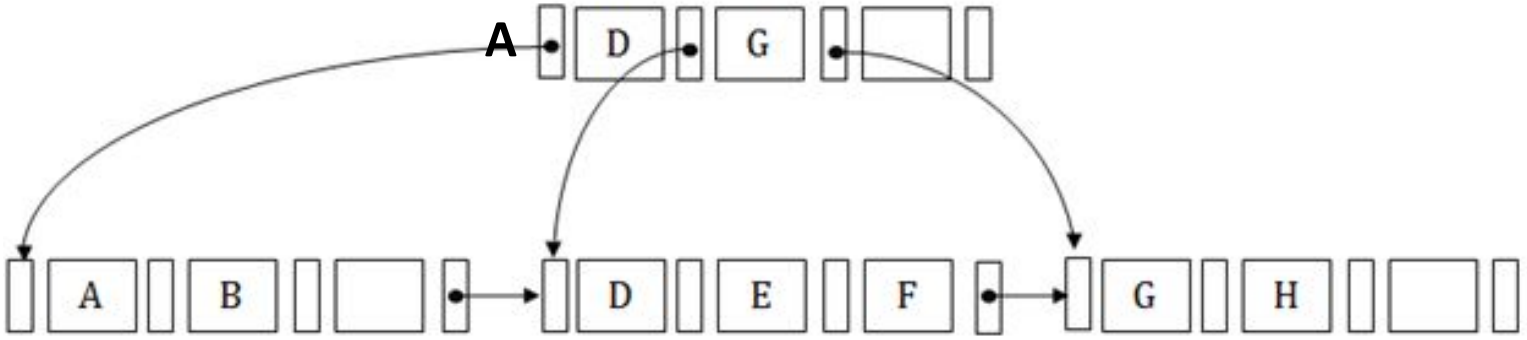
B+ Tree

- The B+ tree is a balanced binary search tree. It follows a multi-level index format.

- In the B+ tree, leaf nodes denote actual data. B+ tree ensures that all leaf nodes remain at the same height.

- In the B+ tree, the leaf nodes are linked using a link list. Therefore, a B+ tree can support random access as well as sequential access.

Structure of B+ Tree



Internal node

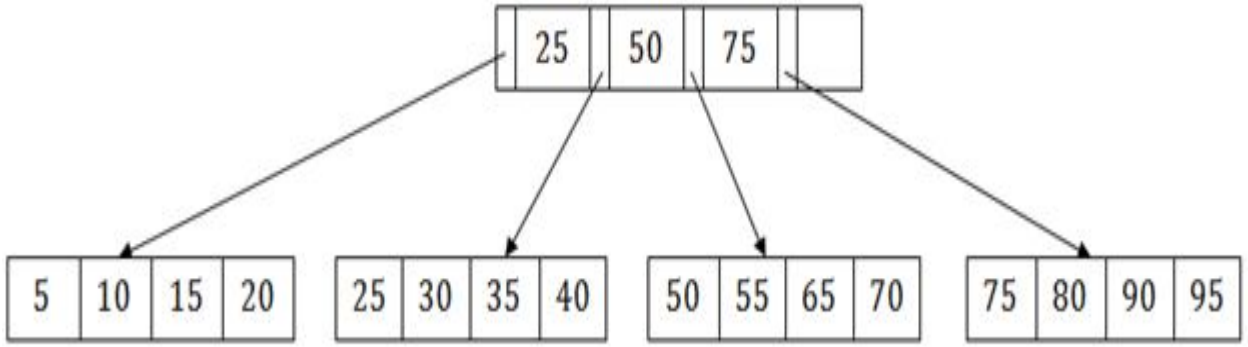
- An internal node of the B+ tree can contain at least n/2 record pointers except the root node.
- At most, an internal node of the tree contains n pointers.

Leaf node

- The leaf node of the B+ tree can contain at least n/2 record pointers and n/2 key values.
- At most, a leaf node contains n record pointer and n key values.
- Every leaf node of the B+ tree contains one block pointer P to point to next leaf node.

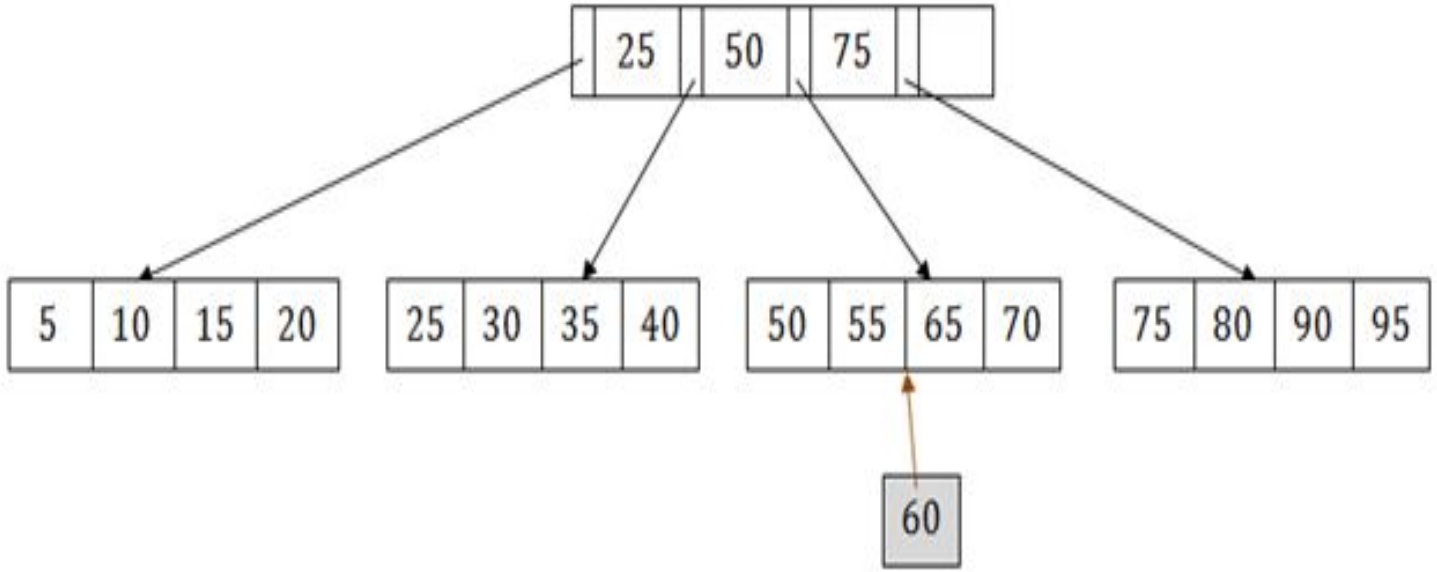
Searching a record in B+ Tree

- Suppose we have to search 55 in the below B+ tree structure.
- First, we will fetch for the intermediary node which will direct to the leaf node that can contain a record for 55.
- So, in the intermediary node, we will find a branch between 50 and 75 nodes
- Then at the end, we will be redirected to the third leaf node. Here DBMS will perform a sequential search to find 55.



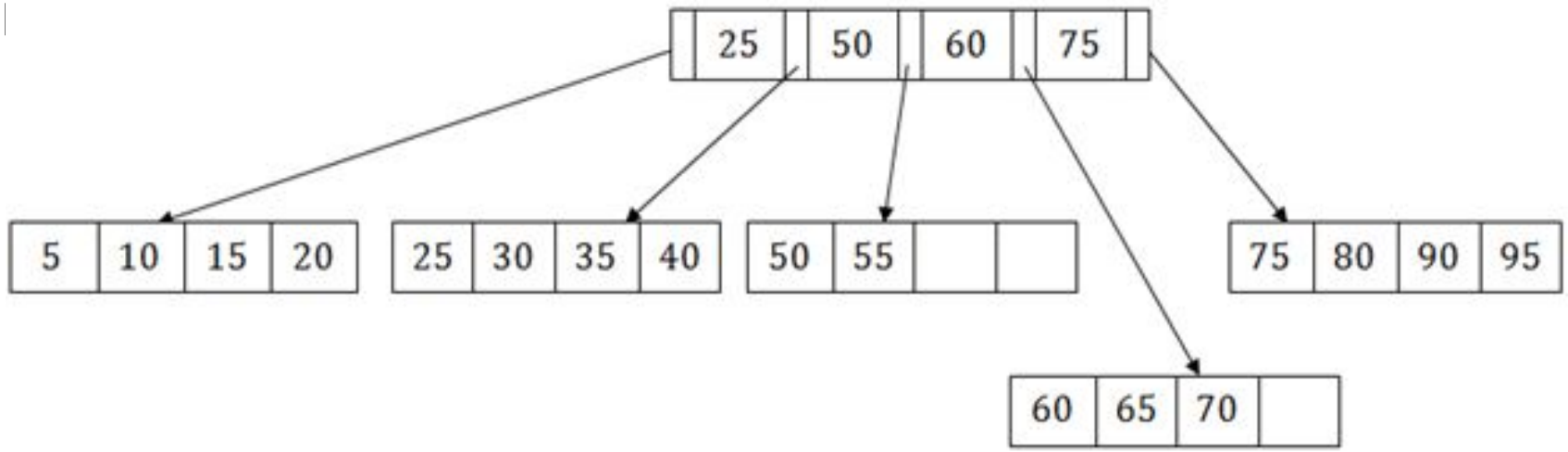
B+ Tree Insertion

- Suppose we want to insert a record 60 in the below structure.
- It will go to the 3rd leaf node after 55.
- It is a balanced tree, and a leaf node of this tree is already full, so we cannot insert 60 there.
- In this case, we have to split the leaf node, so that it can be inserted into tree without affecting the fill factor, balance and order



B+ Tree Insertion

- The 3rd leaf node has the values (50, 55, 60, 65, 70) and its current root node is 50. We will split the leaf node of the tree in the middle so that its balance is not altered. So we can group (50, 55) and (60, 65, 70) into 2 leaf nodes.
- If these two has to be leaf nodes, the intermediate node cannot branch from 50. It should have 60 added to it, and then we can have pointe



- This is how we can insert an entry when there is overflow. In a normal scenario, it is very easy to find the node where it fits and then place it in that leaf node.

B+ Tree Deletion

- Suppose we want to delete 60 from the above example. In this case, we have to remove 60 from the intermediate node as well as from the 4th leaf node too
- If we remove it from the intermediate node, then the tree will not satisfy the rule of the B+ tree. So we need to modify it to have a balanced tree.
- After deleting node 60 from above B+ tree and re-arranging the nodes, it will show as follows:

